Security Self-Assessment — StudentBridge AI

Scope: 2026 FinTechathon, Topic C, competition demonstration. Review date: 18 October 2026.

Status: A student-built planning prototype. No real-money remittance, bank account, licensed payment service, or regulatory certification is claimed.

1. Roles and actual permission boundaries

Role or component	Allowed	Not allowed or not implemented
Anonymous viewer	Access public education sources and available public planning APIs	No verified identity, account or financial authorization
Planning visitor	Calculate fees, review PDF text, compare user-provided hypothetical quotes, export calendar	No authority to execute a real transfer or prove bank eligibility
Sandbox participant	Confirm and record a clearly labeled simulated operation	No real funds move; no authenticated user ownership
Financial engine	Decimal-based deterministic calculations and hard deadline/reserve constraints	Cannot confirm actual provider rates or regulatory permissions
Optional Gemini assistant	Summarize and explain rule-based outputs	Cannot autonomously call real payment APIs or bypass approval
Operator	Manage environment configuration and review local demonstration evidence	Must keep secrets out of public repository and recorded demo

Production restrictions actually present in source code: /api/sandbox/history, /api/audit, and /api/audit/verify return HTTP 403 under APP_ENV=production. These are endpoint-level blocks, not a substitute for full authentication. Other demo endpoints may still contain shared mutable planning state; the public deployment is not a personal finance account system.

2. Implemented controls verified from the code

• FastAPI/Pydantic input validation with constrained request fields and supported currency/country choices.

• Python Decimal financial calculations and source-linked reference exchange-rate handling.

• Check of available funds, living-expense protection (when enabled), deadline and quote expiry before recommending user-entered offers.

• User confirmation phrase and idempotency key required for local sandbox execution only.

• SQLAlchemy queries, MySQL transactions, and HMAC-linked audit records. Changing APP_SECRET can invalidate past audit verification.

• Uploaded PDF type/size/page checks and pypdf text extraction. Candidate fields must be reviewed by the user; this is not authenticated invoice verification or OCR.

• Optional provider-hosted checkout is disabled by default and requires separate authorization; do not enable for a public competition demo.

• APP_ENV=production blocks the shared history/audit endpoints noted above.

3. Risk register — Known gaps

Risk	Priority	What remains to be done
No registered users, authentication or user isolation	Critical before real use	Secure signup, sessions, per-user records, access policies and tests
Previously exposed database/Gemini/app credentials	Critical	Rotate exposed values; remove them from Git history if ever committed; use Render secret settings
Open public prototype APIs, abuse or denial of service	High	Authentication, API limits, bot/file protections, abuse monitoring
Unverified provider quotes and route eligibility	High	Licensed/authorized provider APIs, corridor and compliance verification
Uploaded invoice privacy, forged payee, prompt injection	High	Restricted files, retention/deletion, verified recipients, anti-malware, human review
External FX reference delay/outage	Medium	Freshness indicator and fail-safe 'unavailable' response; never invent executable bank prices
Billing/service uptime for Render / MySQL	Medium	Reliable plans, backups, recovery test, incident plan
Unverified third-party data ownership and licenses	High before public distribution	Contestant review of source terms, redistribution permission and dependency notices

4. Security claims NOT made

No KYC/AML or sanctions checks, bank partnerships, automated fraud detection, production-grade account security, certified cryptographic compliance, or real-money transfers have been independently verified. Sandbox receipts and test logs prove only simulated workflow completion. GET /api/health confirming MySQL access is not evidence of a secure production system.

5. Reproduce key checks

```
python -m pip install -r requirements.txt
python -m pytest -q tests
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Against a separate NON-PRODUCTION MySQL database only:

```
python scripts/check_mysql.py
```

Opt-in additional MySQL integration check, after reviewing its script:

```
MYSQL_SMOKE_TEST=YES python scripts/smoke_test_mysql.py
```

Important: The 76-test suite uses an isolated temporary SQLite database in test mode. It is not an independent live Aiven/Railway/Render/MySQL integration audit.

6. Independent work, privacy and intellectual property

This package keeps original project code, authored UI, source-attribution documentation, and a register of open-source dependencies. The contestants must verify the licenses for the exact resolved dependency

versions and permission to redistribute imported research files. No competition-rules PDF, competitor screenshot, external brand logo or proprietary template is republished in this release. No blanket compliance certification or statement of ownership over third-party material is made.

See docs/ORIGINALITY_AND_LICENSES.md, docs/COMPETITION_SUBMISSION_GUIDE.md, and the official competition website (https://fintechathon.g-ican.com/) for controlling rules and privacy terms.